

Informe de análisis estático de seguridad (SAST)

UniCoin – Iteración de validación de apuntes

11 de junio de 2026

1. Objetivo y alcance

El objetivo del análisis SAST es detectar vulnerabilidades y defectos de seguridad directamente sobre el código fuente, antes de desplegar la aplicación. En UniCoin este control se aplica a la iteración UC-02, que incluye el inicio de sesión, la subida y validación de apuntes, las notificaciones y la asignación de UniCoins y credenciales de token.

El alcance está definido en `sonar-project.properties`: se analizan `app.py` y el directorio `src/`, mientras que las pruebas se encuentran en `tests/`. Se excluyen los ficheros de base de datos, las cachés y `uploads/`, ya que son datos generados y no código fuente. Esta separación reduce ruido y centra los resultados en los componentes mantenidos por el equipo.

2. Herramientas y proceso de análisis

Bandit. Se utiliza como analizador SAST específico para Python y está declarado como dependencia en `requirements.txt`. Su ejecución está integrada en `scripts/check_quality.py` mediante el comando:

```
python -m bandit -r app.py src -x tests,uploads,__pycache__
```

Bandit revisa patrones como secretos escritos en el código, configuraciones inseguras, uso arriesgado de procesos, criptografía débil o construcción insegura de consultas. Durante el desarrollo permitió revisar falsos positivos B105 asociados a nombres de campos y tablas, sin ocultarlos mediante `# nosec`. También se corrigió un hallazgo de configuración: Flask ya no arranca siempre con `debug=True`; el modo depuración solo se activa explícitamente mediante `UNICOIN_DEBUG=1`.

SonarQube. Complementa a Bandit con análisis de vulnerabilidades, *security hotspots*, errores, mantenibilidad y duplicación. El proyecto usa la clave `unicoin-validacion-apuntes` y configura el alcance, versiones de Python y exclusiones en `sonar-project.properties`. La integración contempla dos formas de uso:

- **SonarQube for IDE en PyCharm:** muestra problemas mientras se programa y permite corregirlos antes de ejecutar el análisis completo. Puede utilizarse de forma local o conectarse al proyecto del servidor para compartir reglas.
- **SonarQube Server y SonarScanner:** un servidor local puede levantarse mediante Docker Compose y consultarse en <http://localhost:9000>. Después, `sonar-scanner` envía el análisis y publica el resultado en el panel del proyecto. El metadato de una ejecución previa confirma ese servidor y el panel <http://localhost:9000/dashboard?id=unicoin-validacion-apuntes>.

Durante la verificación se comprobó el despliegue local: el contenedor `unicoin-sonarqube`, creado con la imagen `sonarqube:lts-community`, estaba activo y publicaba el puerto 9000. La API informó del estado UP y de la versión 9.9.8.100196. El fichero Compose se gestiona fuera de esta ruta del repositorio. El script busca primero el scanner incluido en `.tools/sonar-scanner/bin/sonar-scanner` y después uno instalado en el sistema.

3. Automatización y resultados

El punto único de ejecución es:

```
python scripts/check_quality.py
```

El flujo valida primero la configuración de SonarQube, ejecuta las pruebas de regresión, lanza Bandit y, finalmente, ejecuta SonarScanner si está disponible. De esta forma, un fallo funcional o un hallazgo de Bandit detiene el proceso antes de publicar un análisis incompleto. La opción `-require-sonar` permite convertir la ausencia del scanner en error, lo que resulta apropiado para una futura integración continua.

En la verificación realizada el 11 de junio de 2026 se obtuvieron los siguientes resultados:

- **44 pruebas superadas**, incluidas comprobaciones de autorización por rol y asignatura, subida segura, credenciales, contratos y resistencia frente a cargas SQL injection.
- **2.190 líneas de código analizadas por Bandit.**
- **0 hallazgos** de severidad baja, media o alta.
- **0 líneas omitidas con # nsec**, evitando ocultar riesgos sin justificación.

El panel del análisis centralizado del 4 de junio de 2026 muestra el *Quality Gate* como **Passed**. En el código total aparecen **0 bugs**, **0 vulnerabilidades**, **2 code smells**, **1 security hotspot pendiente de revisión**, una deuda técnica estimada de **9 minutos**, **0,0 % de duplicación** y aproximadamente **2,1k líneas Python**. Para el código nuevo, todas las condiciones están superadas: **0 bugs**, **0 vulnerabilidades**, **0 security hotspots**, **0 code smells** y calificación A en fiabilidad, seguridad, revisión de seguridad y mantenibilidad. La cobertura global figura como **0,0 % sobre 724 líneas a cubrir**; por tanto, es necesario publicar un informe de cobertura para incorporar esta métrica al control de calidad. La Figura 1 muestra el resumen global utilizado como evidencia.

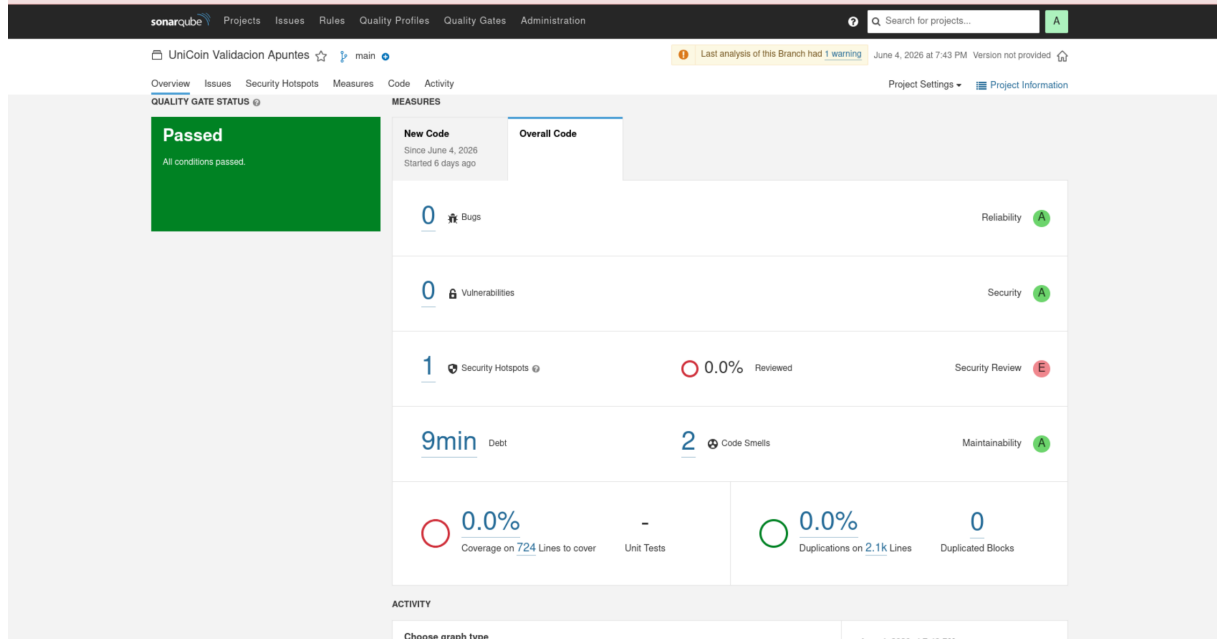


Figura 1: Resultado global del análisis SonarQube: Quality Gate superado, sin bugs ni vulnerabilidades; se observa un security hotspot pendiente, dos code smells y ausencia de cobertura importada.

La ejecución del scanner requiere configurar el servidor y sus credenciales. Los resultados de SonarQube deben validarse en su panel autenticado y no confundirse con el resultado local de Bandit. Además, las pruebas dinámicas y de regresión complementan el SAST, pero no lo sustituyen.

4. Conclusión

UniCoin dispone de un proceso SAST reproducible y adaptado al proyecto: Bandit ofrece una comprobación rápida y obligatoria sobre el código Python; SonarQube for IDE adelanta la detección durante la programación; y el servidor SonarQube, ejecutable localmente mediante Compose, centraliza resultados y reglas de calidad. La última comprobación confirma que el código analizado no presenta hallazgos Bandit y que las 44 pruebas superan los controles de regresión; además, el análisis centralizado supera el Quality Gate sin bugs ni vulnerabilidades. Como mejora, conviene incorporar cobertura, versionar el Compose de SonarQube, revisar el security hotspot pendiente e integrar `check_quality.py -require-sonar` en CI.